Running a Reaction-Diffusion GPU Solver under Nixnan

FP64 vs FP32 Dynamic Range, Exp-Binade Histogramming, and an Exponential `SAMPLING` Sweep

SC26 Tutorial / Nixnan Experiments

1 Executive Summary

This report documents `reaction_diffusion_gpu`, a CUDA port of the 1D reaction-diffusion solver from the FPChecker tutorial (`tutorial/example_2/reaction_diffusion.cpp` in the sibling `SC26-Tutorial-NixNa` repository), run under nixnan (`https://github.com/parfloat/nixnan`) on an NVIDIA RTX 3090. It replaces the repository's prior ad-hoc `rd_nixnan.cu` example (which used its own invented FP16/BF16/FP32 comparison and parameters) with a solver that follows the tutorial's PDE and parameters exactly, and reuses the exp-binade histogram / exponential `SAMPLING` experiment pattern established for `lu_solve_gpu` (`../lu_solve_gpu/lu_solve_gpu_report.tex`). Key results:

1. **FP64 vs FP32 dynamic range**: with $\lambda = 25$ amplifying the solution as $\sim e^{\lambda t}$, the growth factor by $t = 4$ is $e^{100} \approx 2.7 \times 10^{43}$ – inside FP64's range but well past FP32's ($\approx 3.4 \times 10^{38}$). The FP64 build completes cleanly (max 1.70×10^{43} at $t = 4$, zero exceptions); the FP32 build overflows to Infinity between $t = 3.52$ and $t = 3.60$, and the resulting NaN then propagates for the rest of the run – reproducing the CPU tutorial's central lesson ("this case should remain in FP64") on the GPU.

2. **A silent-failure wrinkle**: the GPU port's max-reduction mirrors the CPU version's `std::max_element`, including its NaN-comparison hazard. In this program that hazard goes one step further than the CPU tutorial's own commentary anticipates: it does not just report a misleading value, it makes the reported max *look finite* (literally `0.0`) from the moment of overflow onward, silently defeating the program's own `std::isfinite()` check. nixnan, which instruments individual FADD/FFMA instructions and never looks at the reduction's output, catches the real Infinity and NaN regardless (Section 5).

3. **Same tool-level findings as `lu_solve_gpu` reproduce here**: the `SAMPLING` sweep again shows byte-identical logs across levels $1, 2, 4, 8$, consistent with the key-mismatch bug documented in `../lu_solve_gpu/lu_solve_gpu_report.tex` (Section 7).

4. **Two additional nixnan features**, not exercised in the `lu_solve_gpu` experiment, are demonstrated here: `PRINT_ILL_INSTR+MAX_ERRORS` source/SASS-attributed diagnostics, and `INSTR_MEM` memory instrumentation (Section 8).

2 Provenance and Algorithm

`reaction_diffusion_gpu.cu` solves the 1D linear reaction-diffusion equation

$$\frac{\partial u}{\partial t} = D\,\frac{\partial^2 u}{\partial x^2} + \lambda u, \qquad u(0,t) = u(L,t) = 0, \qquad u(x,0) = \sin\!\left(\frac{\pi x}{L}\right),$$

with an explicit finite-difference (FTCS) update, using exactly the parameters from `tutorial/example_2/reaction_di` $L = 1$, $T = 4$, $N = 101$ spatial points, $D = 0.01$, $\lambda = 25$, $M = 80000$ time steps ($dt = T/M = 5 \times 10^{-5}$).

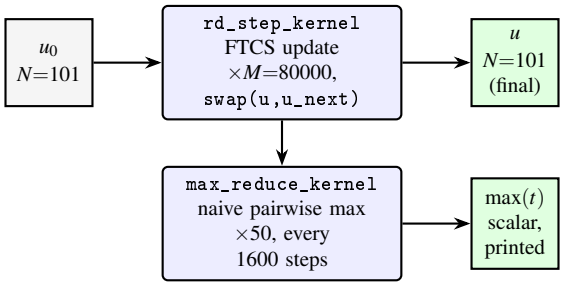
One CUDA kernel (`rd_step_kernel`) performs the update for all interior points in parallel each step; a second kernel (`max_reduce_kernel`) computes the running maximum, launched every $M/50 = 1600$ steps to match the CPU version's print cadence.

Precision is a compile-time macro (`RD_USE_FLOAT`) rather than the CPU tutorial's hand-edited `typedef`, so both variants exist as separate binaries (`reaction_diffusion_gpu_fp64`, `reaction_diffusion_gpu_fp` built from the same source by the `Makefile` – this is what `run_nixnan_experiment.sh` automates instead of requiring a manual edit-and-rebuild cycle per precision.

2.1 Build and Environment

- **GPU**: NVIDIA GeForce RTX 3090, compute capability 8.6
- **Compilation**: `nvcc -std=c++17 -g -O2 -lineinfo -arch=sm_86`
- **Instrumentation**: `LD_PRELOAD=../../nixnan.so`

3 Workflow Diagram: `reaction_diffusion_gpu.cu`



Single time-stepping kernel (`rd_step_kernel`) advances the whole interior of u one FTCS step at a time, ping-ponging between two device buffers (`d_u`/`d_u_next`, swapped by pointer after every launch) for M=80000 steps. Every 1600 steps (`print_interval = M/50`), a side branch launches `max_reduce_kernel` on the *current* u buffer to compute and print the running maximum — this is the naive `v > best` reduction discussed in Section 5, not part of the main time-stepping recurrence.

4 FP64 vs FP32: Dynamic Range

	FP64	FP32
Max $\|u\|$ at $t = 3.52$	1.11×10^{38}	1.10×10^{38}
Max $\|u\|$ at $t = 3.60$	8.09×10^{38}	`0.0` (see §5)
Max $\|u\|$ at $t = 4.00$	1.70×10^{43}	`0.0`
First non-finite reported by the program itself	never	never (masked)
FP32/FP64 exceptions (nixnan, Stage 1)	0/0/0	36 NaN / 16 Inf / 40 −Inf

Table 1: From `logs/baseline/{fp64,fp32}.log` and `logs/histogram/{fp64,fp32}.log`. FP32's reported maxima are the reduction's output, not the true array contents – see Section 5.

Both builds agree closely up through $t = 3.52$ (max $\approx 1.1 \times 10^{38}$, just below FP32's ceiling of $\approx 3.4 \times 10^{38}$). By the next printed sample, $t = 3.60$, FP32 has overflowed to Infinity while FP64 continues to grow smoothly; FP64 finishes the full simulation at 1.70×10^{43}, comfortably inside its $\approx 1.8 \times 10^{308}$ range. This is exactly the tutorial's intended lesson, now demonstrated on the GPU: a workload that "looks fine" while values stay under $\sim 10^{38}$ becomes numerically unsafe in FP32 the moment it doesn't.

5 A Silent-Failure Wrinkle in the GPU Port

The CPU version's `print_max_value` uses `std::max_element`, whose pairwise comparisons are all `false` once a NaN is involved (a NaN is neither greater than nor less than anything, including itself). `max_reduce_kernel` in this port intentionally mirrors that logic (`v > best`) rather than "fixing" it with e.g. `fmax`, in order to carry the same authentic hazard onto the GPU.

In this program, however, the hazard manifests more severely than a merely misleading value. `logs/baseline/fp32.log` shows the reported maximum becoming exactly

```
At time t = 3.600000, Maximum value of u = 0.0000000000e+00
At time t = 3.680000, Maximum value of u = 0.0000000000e+00
...
At time t = 4.000000, Maximum value of u = 0.0000000000e+00
```

from $t = 3.60$ onward – not `nan`, not `inf`, but a literal, plausible-looking, finite `0.0`. The mechanism: the reduction seeds `best` from `u[0]` (a boundary point, pinned to exactly `0.0` by the Dirichlet condition) and only updates it via `v > best`. Once the array's interior values become Infinity/NaN, none of those comparisons succeed, so `best` never leaves its seed value of `0.0`.

This has a second-order consequence: `reaction_diffusion_gpu.cu` also contains a self-check –

```
if (!reported_nonfinite && !std::isfinite((double)max_val))
{
    reported_nonfinite = true;
    std::cerr << "first non-finite reported max at step " << ...
}
```

This message never appears in `logs/baseline/fp32.log`'s stderr, because `max_val` itself is `0.0` – perfectly finite by the check's own definition – even though the array it was reduced from genuinely contains Infinity and NaN. A monitoring strategy built entirely on top of this program's own reported summary statistic would conclude, incorrectly, that nothing ever went wrong.

nixnan is unaffected by this because it does not consume the reduction's output at all: it instruments every `FADD`/`FFMA` as it executes, inside `rd_step_kernel` itself, before the corrupted value ever reaches `max_reduce_kernel`. `logs/histogram/fp32.log` and `logs/memory/fp32.log` both report the real counts (36 NaN, 16 Infinity, 40 −Infinity in registers; 1 NaN value, 36,928 repeats, actually written to global memory) that the program's own console output never shows. This is, in miniature, the argument for binary-level floating-point exception instrumentation: a program's own summary statistics can be fooled by exactly the kind of comparison logic that silently ignores non-finite values, while instruction-level instrumentation cannot.

6 Stage 1: Exp-Binade Histogram Gathering

`bin_spec.json` (generated by `gen_bin_spec.py`) populates *both* `f32` and `f64` – unlike `lu_solve_gpu`, where only one precision existed per build, here both formats are genuinely present across the two binaries that share this one spec file. Each format's full valid exponent range is tiled in buckets of width 4 (the same "bucket size 4" convention as `../lu_solve_gpu/gen_bin_spec.py`): `f32` $\in [-126, 127]$ (64 buckets), `f64`$\in [-1022, 1023]$ (512 buckets). `f16`/`bf16` are left empty; this program never uses them.

The report-threshold `count` required re-tuning for this workload. $M = 80000$ steps over $\sim$99 interior points produce roughly 1000$\times$ more instrumented FP-op instances than `lu_solve_gpu`'s one-shot 20×20 solve; reusing `lu_solve_gpu`'s `count: 128` produced $\sim$950,000 report lines here (Table 2). `count: 1,000,000` was chosen empirically to land back in the same "nice" $\sim$100–150-reports-per-run range.

count	FP64 reports	FP32 reports
128	1,050,603	943,683
100,000	1,339	1,203
1,000,000 (chosen)	121	109

Table 2: Report-line volume for a single run at three candidate `count` thresholds (measured during tuning; not stored under `logs/`).

7 Stage 2: Exponential `SAMPLING` Sweep

The same schedule as `../lu_solve_gpu/run_nixnan_experiment.sh` was applied to `reaction_diffusion_gpu_f` `SAMPLING` $\in \{1,2,4,8\}$, 16 process launches ("snaps") per level, then reset back to 1 – 64 whole-program runs, stored under `logs/sampling/cycle_01/sampling_{1,2,4,8}/snap_{01..16}`. `rd_step_kernel` is launched 80,000 times per run, so `SAMPLING` has far more opportunity here than in `lu_solve_gpu` (19 launches per kernel) to visibly change instrumentation coverage if it worked. It still does not: `sampling_1/snap_01.log` and `sampling_8/snap_01.log` are byte-identical, consistent with the key-mismatch bug documented in `../lu_solve_gpu/lu_solve_gpu_report.tex` (Section 4.2 there: `should_instrumen` looks up the per-kernel invocation counter by the full mangled signature, while `nvbit_at_cuda_event()` increments it under the name truncated by `cut_kernel_name()`, so the two keys never match). This report does not re-derive that finding in full; see the earlier report for the code excerpts.

8 Stage 3: Interesting Exception Settings

Two nixnan features not exercised in the `lu_solve_gpu` experiment:

8.1 3a: `PRINT_ILL_INSTR` + `MAX_ERRORS`

Run with `PRINT_ILL_INSTR=1 MAX_ERRORS=20`, nixnan prints the exact offending SASS instruction for each exception and terminates the process once 20 have been reported. On `reaction_diffusion_gpu_fp32` this cap is reached almost immediately after the overflow at $t \approx 3.6$; of the 20 reported errors, 14 are attributed to `reaction_diffusion_gpu.cu:64` (the diffusion-term computation, `D_dt_over_dx2 * (u[i+1] - 2*u[i] + u[i-1])`) and 6 to `reaction_diffusion_gpu.cu:66` (the state update, `u_next[i] = u[i] + diffusion_term + reaction_term`) – consistent with $\text{Inf} - 2 \cdot \text{Inf} + \text{Inf} =$ NaN occurring right where the stencil combines three already-overflowed neighbors. `reaction_diffusion_gpu_fp64` never reaches the cap and runs to completion (`logs/diagnostic/fp64.log` has zero error lines).

8.2 3b: `INSTR_MEM`

Run with `INSTR_MEM=1`, nixnan additionally instruments memory store instructions. `logs/memory/fp32.log` shows

```
#nixnan: --- FP32 Memory Operations ---
#nixnan: NaN: 1 (36928 repeats)
```

confirming that NaN is not merely a transient register value but is actually written to `u_next` in GPU global memory, repeatedly, for the remainder of the run (80,000 total steps, overflow first appears around step 72,000 at $t \approx 3.6$, so on the order of $8000 \times 99 \approx 800{,}000$ store opportunities remain – 36,928 of the distinct memory NaN events were captured). `logs/memory/fp64.log` shows zero, as expected.

9 Conclusions

1. The GPU port reproduces the CPU tutorial's central lesson exactly: FP64 is safe for this workload's full time horizon; FP32 is not, and fails via the textbook overflow-then-NaN-propagation path.

2. The port's naive max-reduction (chosen deliberately to mirror `std::max_element`'s NaN hazard) turns out to hide the FP32 overflow from the program's own console output and its own finiteness self-check – a instructive illustration of why instruction-level exception detection is valuable independent of whatever a program's own diagnostics report.

3. The `SAMPLING` key-mismatch bug found while working on `lu_solve_gpu` reproduces here under a workload with 1000× more kernel launches, reinforcing that it is a property of nixnan itself and not an artifact of the smaller LU experiment.

4. `PRINT_ILL_INSTR+MAX_ERRORS` and `INSTR_MEM` both worked as documented and produced exactly the kind of source-line- and memory-level evidence the CPU tutorial's own `fpc-create-report` is meant to provide, here on the GPU build.

Reproduction

```
cd ~/repos/nixnan/SC26-expts/rd_nixnan
./run_nixnan_experiment.sh
```

Requires a built `../../nixnan.so`. See `README.md` in this directory for file-by-file documentation and override variables (`NIXNAN_SO`, `CYCLES`, `SAMPLING_EXE`).